



- Scope for future enhancements such as persistent storage and account security.

7. Core Logic

7a. Core Logic - Program 1: Electricity Bill Calculator

The `calc()` function is the core of the program. It determines the bill using slab-wise rates, applies a threshold-based discount, and adds a fixed charge:

```
def calc(unit):
    bill = 0
    discount = 0
    charge = FIXED_CHARGE

    # Slab-wise tariff: rate increases as consumption rises
    if unit <= 100:
        bill = unit * 1.5
    elif unit <= 200:
        bill = (100 * 1.5) + ((unit - 100) * 2.5)
    elif unit <= 300:
        bill = (100 * 1.5) + (100 * 2.5) + ((unit - 200) * 4.0)
    else:
        bill = (100 * 1.5) + (100 * 2.5) + (100 * 4.0) + ((unit - 300) * 6.0)

    # 10% discount applies only above a Rs. 1000 threshold
    if bill > 1000:
        discount = DISCOUNT_RATE

    amount_after_discount = bill - (bill * discount) / 100
    final_bill = amount_after_discount + charge # fixed charge added last

    return final_bill
```

In short: the function accumulates cost slab by slab as units increase, applies a 10% discount only when the pre-charge bill exceeds Rs. 1000, and finally adds the fixed Rs. 50 service charge before returning the total.

7b. Core Logic - Program 2: ATM Transaction Simulator

The authentication and withdrawal routines form the operational core of the simulator. `authenticate()` gatekeeps every transaction by verifying the card number and PIN before any account data is accessed, while `withdraw()` enforces that a transaction cannot exceed the available balance.

```
def authenticate(card_no, pin):
    if card_no in ACCOUNTS and ACCOUNTS[card_no]["pin"] == pin:
        return True # credentials match
    return False # invalid card/PIN

def withdraw(card_no, amount):
    if amount <= ACCOUNTS[card_no]["balance"]:
        ACCOUNTS[card_no]["balance"] -= amount # deduct funds
        return True
    return False # insufficient balance
```